FINAL PROJECT REPORT

STUDY SPRINT

A desktop study-planning application

Course: Software Design Patterns

Platform: Java 17, Java Swing

Team: Erasyl, Arsen, Abylaikhan, Adilzhan

Academic year: 2026-2027

Repository: [insert the team's actual GitHub link before submission]

Submission note. This document describes the current working draft. The four participants must review their contributions and add their real Git development history before submitting the project.

1. Introduction

Many students have several assignments to complete before exams. Some tasks are more important than others, while some require less time. Study Sprint is a small desktop application that helps users organize these tasks and make a simple study schedule.

The application is designed for an individual student. It works without an Internet connection and stores its data in a local file. We chose this topic because the main functions are easy to demonstrate, and design patterns can be connected to real user actions.

2. Goals and main user journey

The user creates or selects a plan, adds Reading or Practice tasks, chooses a scheduling method, generates a schedule, checks the result and saves a report. Tasks and plan settings remain saved after restarting the application.

3. Domain model

Entity	Role in the project
StudyPlan	Contains plan name, settings, tasks and generated sessions.
StudyTask	Stores title, subject, duration, priority, notes and completion state.
StudySession	Stores the ordered position and the ID of a planned task.
ReadingTask / PracticeTask	Two concrete task types with different study tips.

4. User scenarios

ID	Action	Expected outcome
S1	Create or rename a plan	The selected plan is updated.
S2	Add, edit, complete or delete a task	The task list and stored data change.
S3	Choose priority/shortest and optional breaks/tips	The selected options are saved.
S4	Generate schedule	Ordered tasks and total time are displayed.
S5	Import CSV file	Old-format tasks appear in the plan.
S6	Save summary/detailed TXT or HTML	An exported file is produced.
S7	Restart application	Saved plans and tasks are restored.
S8	Enter invalid values	An explanation is shown; invalid data is rejected.

5. User interface and example result

The Java Swing interface contains a plan selector, task table, task editor buttons, schedule options, a result area and an activity history. The user does not need to operate the program from the command line.

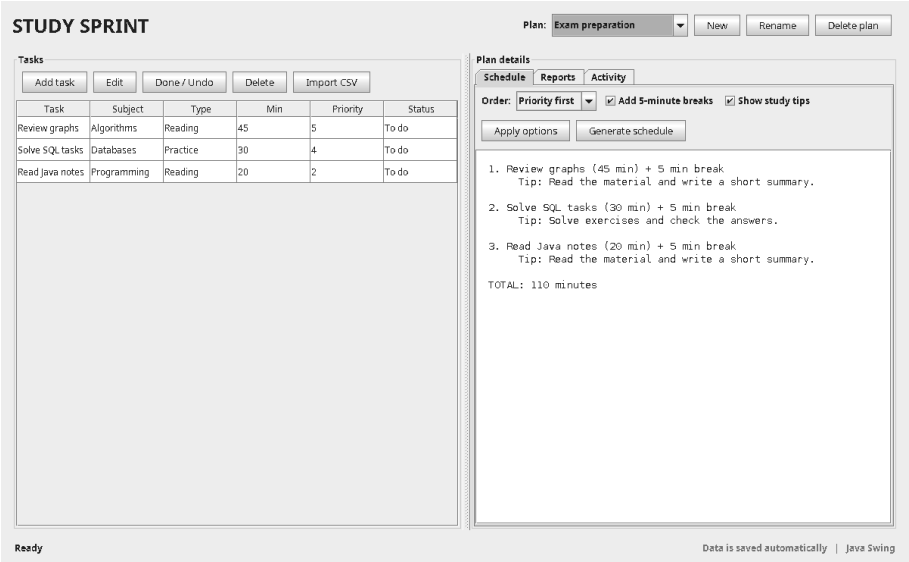


Figure 1. Main window with three example tasks and a generated study schedule. The screenshot is shown in grayscale for printing.

Example: the three sample tasks last 45, 30 and 20 minutes. With a 5-minute break after each task, the displayed total is 110 minutes.

6. Program architecture

Layer	Main classes	Responsibility
Interface	MainFrame, TaskDialog	Forms, buttons, tables and messages.
Application	StudyFacade	Provides operations for the interface and coordinates components.
Business logic	ScheduleService, strategies, creators, decorators	Builds schedules and task descriptions.
Model	StudyPlan, StudyTask, StudySession	Represents the application data.
Storage	PlanRepository	Loads and saves the plan file.

The interface is separated from storage and scheduling rules. For example, a GUI button calls a method on StudyFacade; the facade then uses ScheduleService or PlanRepository. This makes the main methods easier to locate during the defense.

7. Design patterns used

The assignment requires at least six different design patterns, including creational, structural and behavioral patterns. Our application currently uses eight patterns. Each of the four assigned members has two patterns to study, demonstrate and independently adapt.

Pattern	Problem and main classes	How we demonstrate it	Assigned member
Builder	Create StudyPlan with several options. StudyPlanBuilder.	Create a new plan and change settings.	Erasyl
Strategy	Select task ordering. ScheduleStrategy, PriorityStrategy, ShortestStrategy.	Compare the two scheduling methods.	Erasyl
Factory Method	Create different tasks. TaskCreator, ReadingTaskCreator, PracticeTaskCreator.	Create Reading and Practice tasks.	Arsen
Decorator	Add optional information and break time. SessionText, BreakDecorator, TipDecorator.	Toggle breaks and tips.	Arsen
Observer	Notify after a plan change. PlanObserver, ActivityHistory, MainFrame.	Edit a task and inspect Activity.	Abylaikhan
Facade	Keep complex services behind one object. StudyFacade.	Trace a GUI button to the facade and storage.	Abylaikhan
Adapter	Import an old CSV format. TaskImporter, LegacyCsvAdapter, OldCsvReader.	Import legacy_tasks.csv.	Adilzhan
Bridge	Combine two report types with TXT/HTML formats. StudyReport, ReportFormat.	Export detailed HTML and summary TXT.	Adilzhan

Pattern categories

Creational: Builder and Factory Method. **Structural:** Decorator, Facade, Adapter and Bridge. **Behavioral:** Strategy and Observer.

The application uses the patterns in its normal workflows. The classes are not included only to satisfy the required count.

How the parts work together

A plan is created through Builder. Factory Method creates tasks. Strategy determines their order and Decorator changes session text and break duration. Observer reacts when the plan changes. Facade connects user actions to the services. Adapter imports tasks and Bridge exports reports.

8. Pattern implementation examples (part 1)

8.1 Builder - Erasyl

The builder collects optional settings and creates the plan in one place. This keeps the plan creation code readable.

```
StudyPlan plan = new StudyPlanBuilder()
    .name("Exam preparation")
    .strategy("Priority first")
    .breaks(true).tips(true).build();
```

8.2 Strategy - Erasyl

The scheduling service chooses an ordering algorithm through the same interface. Replacing the ordering algorithm does not require changing the task classes.

```
ScheduleStrategy strategy = "Priority first".equals(plan.getStrategy())
    ? new PriorityStrategy() : new ShortestStrategy();
List<StudyTask> tasks = strategy.arrange(plan.getTasks());
```

8.3 Factory Method - Arsen

The abstract creator provides a common way to make tasks; the concrete creator decides which task subclass to return.

```
public class ReadingTaskCreator extends TaskCreator {
    @Override
    public StudyTask makeTask(String id, String title, String subject,
        int minutes, int priority, String notes) {
        return new ReadingTask(id, title, subject, minutes, priority, notes);
    }
}
```

8.4 Decorator - Arsen

The session object can be wrapped by optional features. One option adds break time and another adds study advice.

```
SessionText session = new BasicSessionText(task);
if (plan.hasBreaks()) session = new BreakDecorator(session);
if (plan.hasTips()) session = new TipDecorator(session, task);
```

Limitation: the scheduling strategies are intentionally simple. They sort tasks but do not calculate real calendar dates or detect timetable conflicts.

9. Pattern implementation examples (part 2)

9.1 Observer - Abylaikhan

When the facade changes data, it saves the plans and informs registered observers. The activity history and interface can react to the change.

```
for (PlanObserver observer : observers) {
    observer.onPlanChanged(message);
}
```

9.2 Facade - Abylaikhan

The main window uses simple public facade methods instead of handling file saving and scheduling directly.

```
app.addTask(type, title, subject, minutes, priority, notes);
app.setOptions(strategy, breaksEnabled, tipsEnabled);
app.generateSchedule();
String preview = app.getReport(true, true);
```

9.3 Adapter - Adilzhan

The adapter converts rows from the old reader into StudyTask objects understood by the application.

```
for (String[] cells : oldReader.readRows(file)) {
    results.add(TaskCreators.forType(cells[4].trim()).create(
        cells[0], cells[1], Integer.parseInt(cells[2].trim()),
        Integer.parseInt(cells[3].trim()), "Imported from CSV"));
}
```

9.4 Bridge - Adilzhan

The report detail level and output format are independent. Two report types can be combined with TXT or HTML without duplicating four report classes.

```
ReportFormat format = html ? new HtmlFormat() : new PlainTextFormat();
StudyReport report = detailed
    ? new DetailedReport(format) : new SummaryReport(format);
return report.create(current, scheduler);
```

The UML class diagram in Appendix A shows pattern participants. Appendix B shows the Generate Schedule sequence.

10. Testing

We use a small Java test runner that has no external dependencies. The log in **docs/test_output.txt** reports **ALL 22 TESTS PASSED**. The scenarios below combine related automated checks.

No.	Test input / action	Expected result	Actual
1	First launch, new data file	3 sample tasks are created	PASS
2	Create plan with a name	Plan is saved and selected	PASS
3	Add Reading and Practice	Different task types and tips	PASS
4	Switch Priority and Shortest	Task order changes	PASS
5	Use breaks and tips	Added break time and study advice	PASS
6	Generate schedule	Observer receives update	PASS
7	Complete a task	Old schedule is cleared	PASS
8	Reload stored data	Completion is preserved	PASS
9	Generate TXT and HTML	Both output formats work	PASS
10	Enter empty title	Rejected with error	PASS
11	Enter duration = 1	Rejected, minimum is 5	PASS
12	Enter priority = 8	Rejected, maximum is 5	PASS
13	Enter empty plan name	Rejected with error	PASS
14	Import valid CSV rows	Two tasks are imported	PASS
15	Import invalid CSV row	No partial import	PASS
16	Edit, delete and reopen	Changes remain saved	PASS

These results test business logic. The user interface was opened and a screenshot was captured, but every team member should also manually test GUI buttons on the machine used for the defense.

11. Validation and error handling

The application rejects empty plan names, task titles and subjects. A task must have a duration from 5 to 300 minutes and a priority from 1 to 5. It does not allow generating a schedule with no unfinished tasks or deleting the last remaining plan. Changes that make a schedule outdated clear the old sessions. An invalid CSV is rejected before its rows are added.

12. SOLID principles and code structure

Principle	How it appears in Study Sprint
Single Responsibility	MainFrame handles GUI; ScheduleService handles schedules; PlanRepository handles files.
Open / Closed	New strategies and formats can be added through existing interfaces.
Liskov Substitution	ReadingTask and PracticeTask can be used as StudyTask objects.
Interface Segregation	Small contracts such as TaskImporter and ScheduleStrategy.
Dependency Inversion	Scheduler and report logic depend on interfaces; construction inside facade is a simple trade-off.

13. How to run the application

Requirements: JDK 17 or later, IntelliJ IDEA or a terminal, and a desktop environment.

Action	Instructions
Run in IntelliJ	Open project folder, select JDK 17, run kz.studysprint.App.
Run on Windows	Double-click run.bat.
Run on Linux/macOS	Execute ./run.sh.
Run tests	Use test.bat or ./test.sh.
Import sample tasks	Use samples/legacy_tasks.csv.
Save and reopen	Plans are stored in data/plans.txt on the local computer.

The source files are in **src/main/java**; automated checks are in **src/test/java**. No database account or external framework is required.

14. Proposed responsibility division

The following table is the planned assignment of work to four members. It must be checked against real coding, testing and Git commits before the team claims individual authorship.

Member	Assigned patterns	Files / responsibilities	Suggested live change
Erasyl	Builder, Strategy	StudyPlanBuilder; strategies; plan options and schedule order.	Add Longest first.
Arsen	Factory Method, Decorator	TaskCreator and subclasses; session wrappers.	Add one task variant or decorator.
Abylaikhan	Observer, Facade	StudyFacade; PlanObserver; ActivityHistory; MainFrame integration.	Add a new subscriber.
Adilzhan	Adapter, Bridge	LegacyCsvAdapter; import; Summary/Detailed; HTML/TXT formats.	Add MarkdownFormat.

15. Development plan and evidence

Week	Required work	Evidence to show
1	Problem, scenarios, sketch, initial UML, runnable skeleton.	Actual progress check and commits.
2	Working GUI journey, all patterns, local storage.	Real demo and implementation commits.
3	Validation, tests, diagrams, report and defense practice.	Final code review and instructor update.

The course brief requires progress demonstrations at the end of each week, with submission by the end of Week 9 and individual defense in Week 10. This report does not claim that any progress meetings have already happened.

16. Individual defense

Each member should demonstrate both assigned patterns in code and in the running GUI, explain the problem they solve and their limitations, and complete a small code change suggested by the instructor. All four members should be able to explain the basic workflow, domain model, file saving and exception handling.

17. Limitations

Study Sprint is intended for one student and stores data on one computer. It has no accounts, shared editing or cloud synchronization. The CSV reader expects a simple five-column format without quoted commas. Scheduling uses basic ordering rather than deadline-aware planning. The tests concentrate on business logic, so manual GUI verification remains important.

18. Future work

Possible additions include a weekly calendar, deadline-based strategies, task search, reminders, better CSV parsing, JUnit tests and optional cloud backup. These are ideas for future versions, not features of the current implementation.

19. Conclusion

Study Sprint demonstrates an end-to-end study-planning workflow using Java Swing. Students can manage tasks, choose between two schedules, see notifications, restart without losing data and save a report. Eight design patterns are connected to these functions. The project is deliberately small enough for the four team members to understand and extend during the defense.

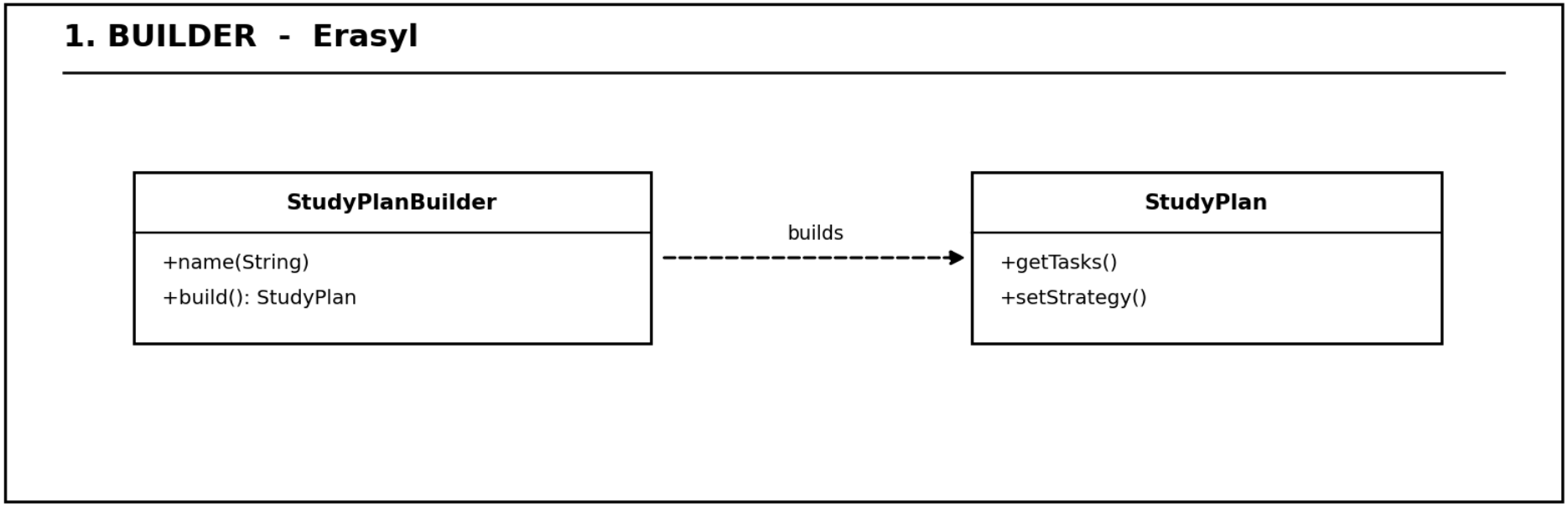
20. External resources and AI use

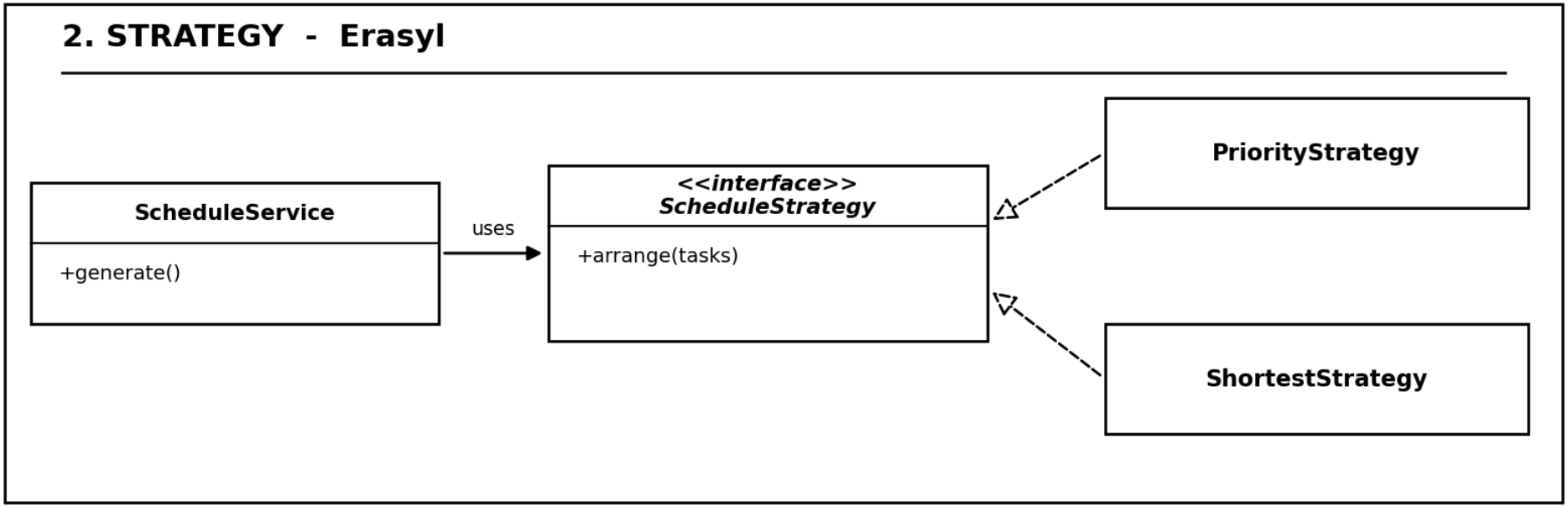
This initial implementation, report and diagram drafts were prepared with assistance from OpenAI ChatGPT. Java language and Swing standard-library concepts were also used. No paid service or additional framework is required. The final team should accurately state which parts were independently reviewed or modified by each member. The course brief allows AI for learning support but does not allow it to replace independent development.

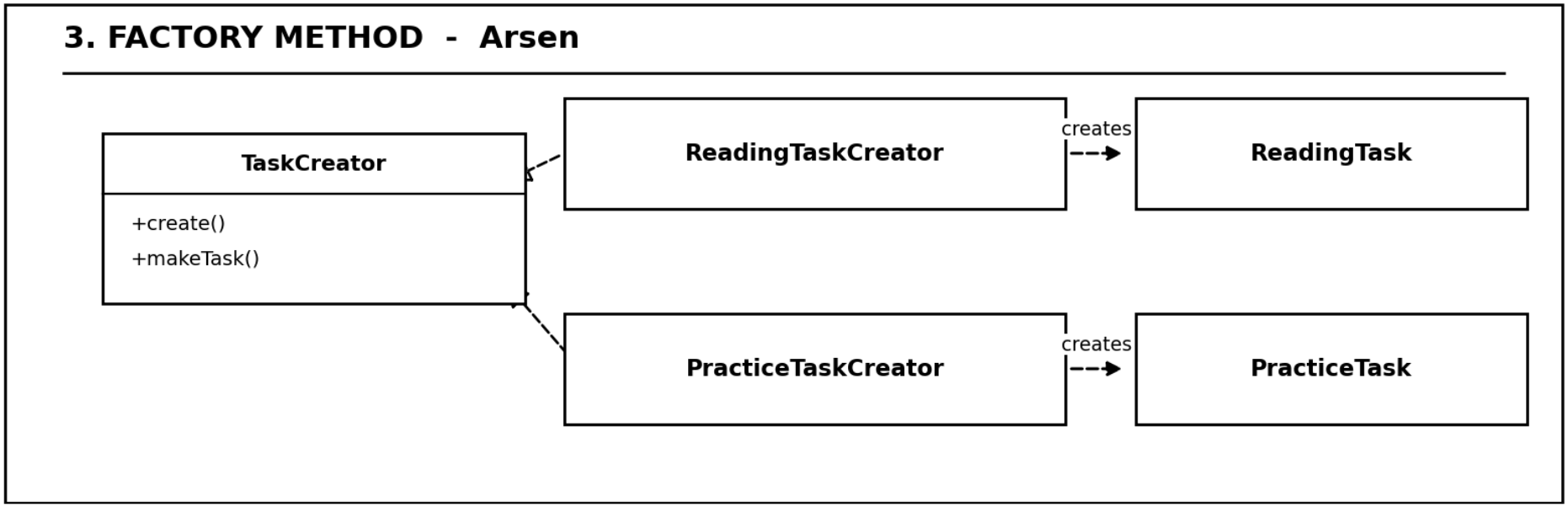
21. Submission checklist

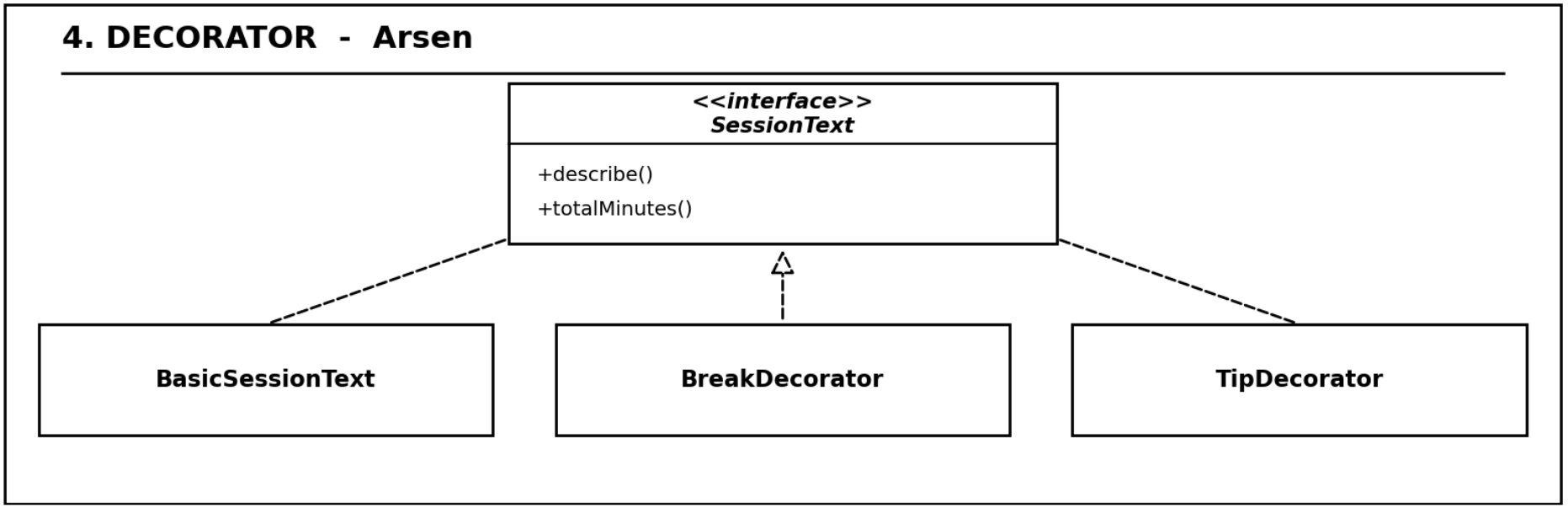
Item	Status before Moodle submission
Source code / ZIP	Included in the project archive.
README and setup	Included, with launch and test instructions.
UML class / sequence	Appendices A and B; separate PNG/SVG files.
Screenshots	Grayscale screenshot of running GUI.
Test results	22 automated checks in docs/test_output.txt.
GitHub/GitLab link	To be added from the real team repository.
Independent contributions	To be verified by students and actual commits.
Individual defense	Each person must prepare the live demonstration.

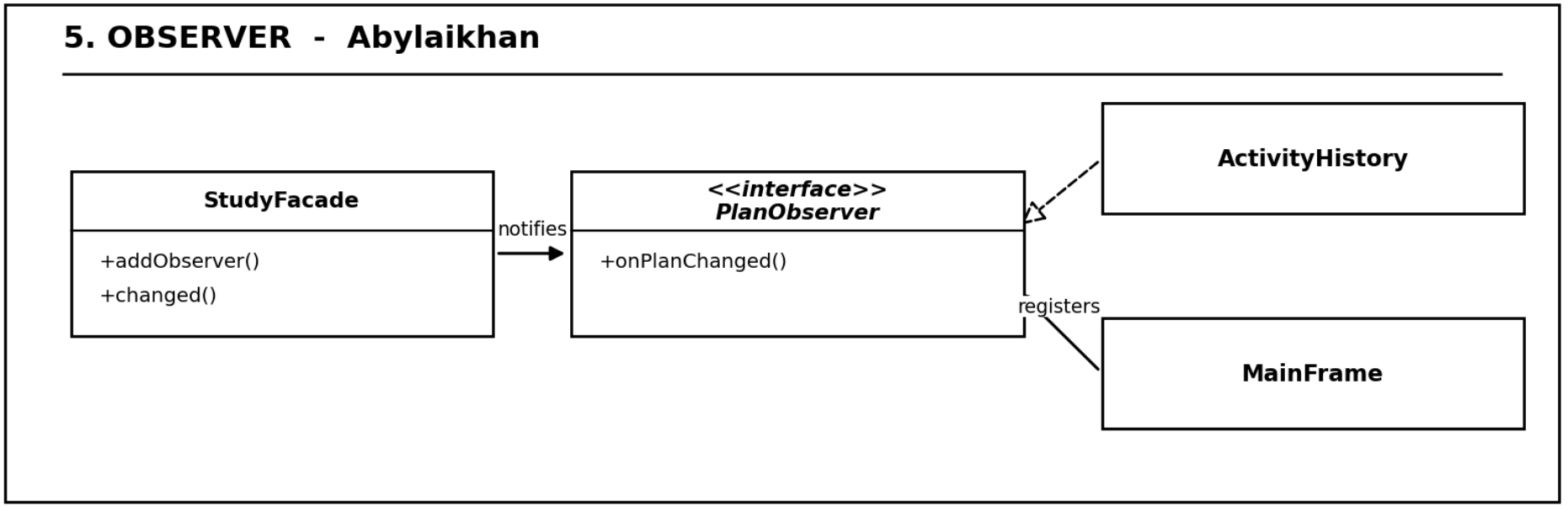
Appendices: A - Class diagram (8 pattern groups). B - Sequence diagram (Generate Schedule).

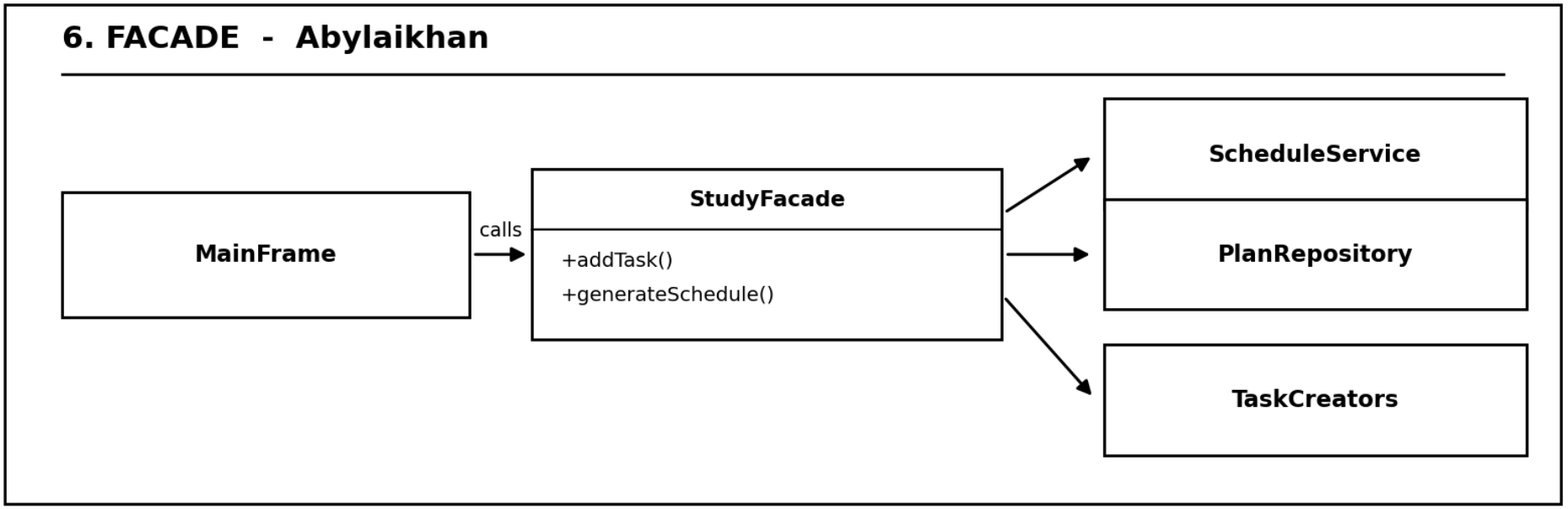

1. BUILDER - Erasyl
StudyPlanBuilder
+name(String)
+build(): StudyPlan
builds
StudyPlan
+getTasks()
+setStrategy()

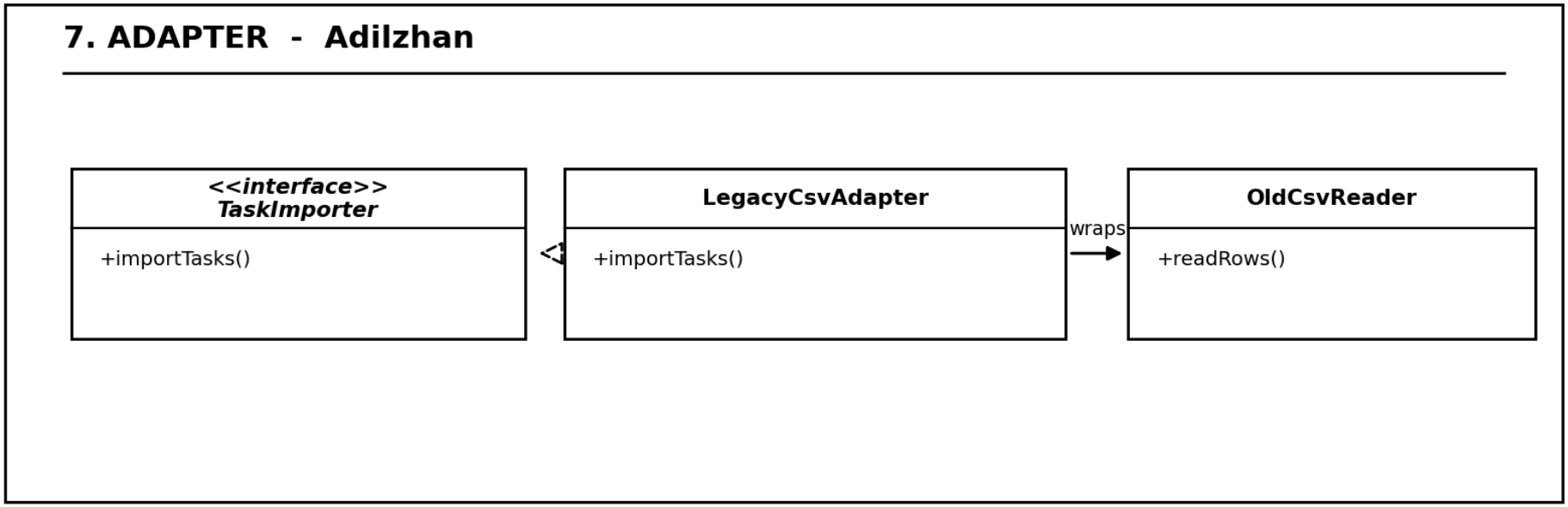

2. STRATEGY - Erasyl
ScheduleService
+generate()
uses
<<interface>>
ScheduleStrategy
+arrange(tasks)
PriorityStrategy
ShortestStrategy

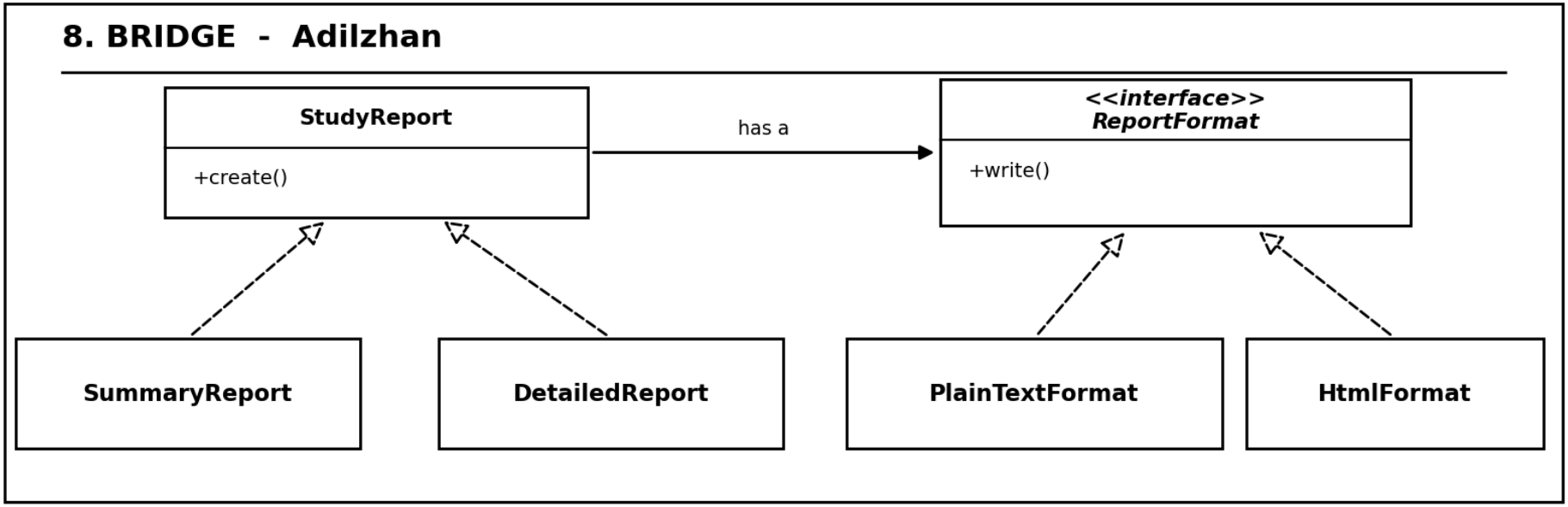

3. FACTORY METHOD - Arsen
TaskCreator
+create()
+makeTask()
ReadingTaskCreator
creates
ReadingTask
PracticeTaskCreator
creates
PracticeTask


4. DECORATOR - Arsen
<<interface>>
SessionText
+describe()
+totalMinutes()
BasicSessionText
BreakDecorator
TipDecorator

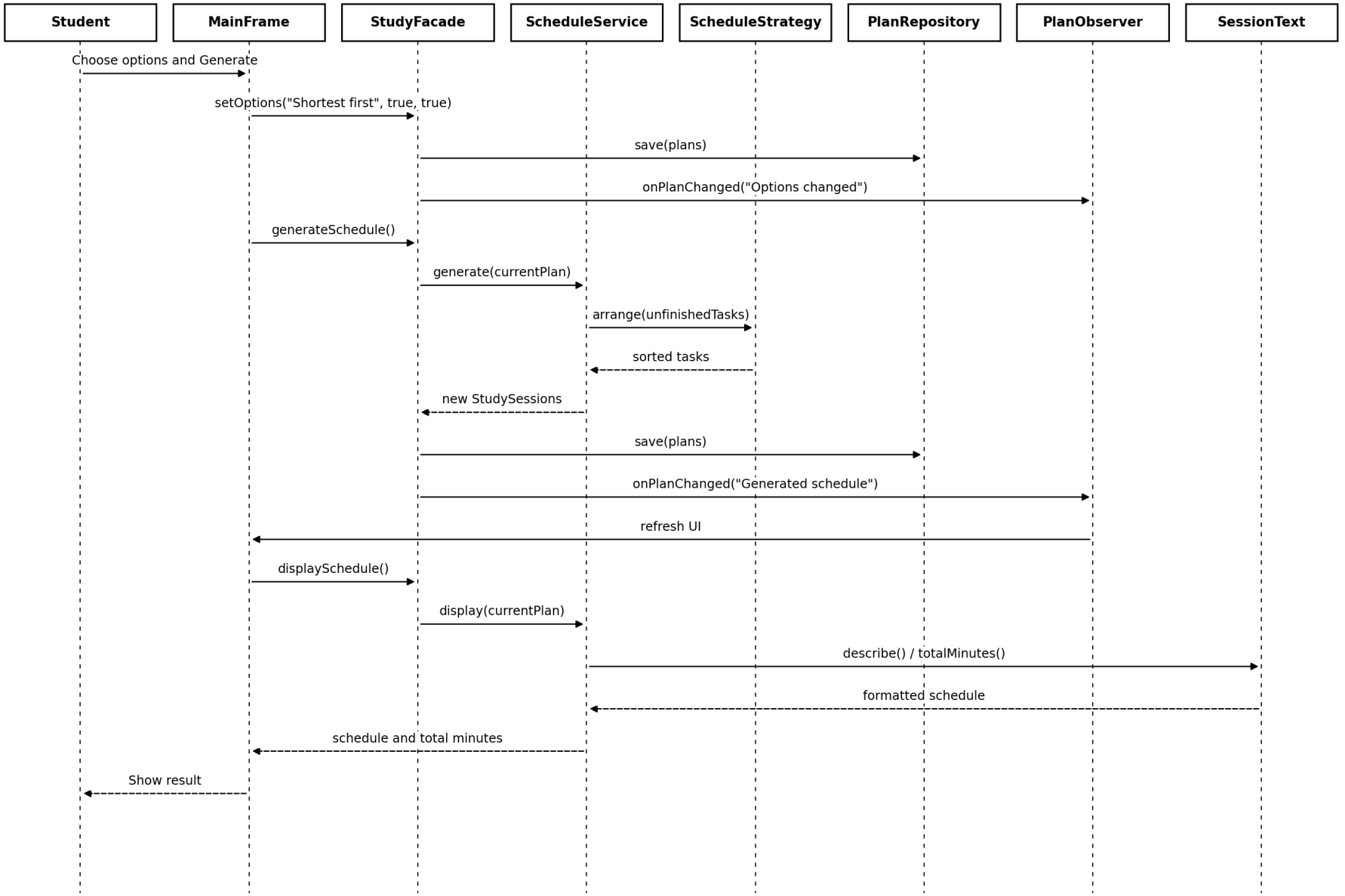

5. OBSERVER - Abylaikhan
StudyFacade
+addObserver()
+changed()
notifies
<<interface>>
PlanObserver
+onPlanChanged()
ActivityHistory
registers
MainFrame


6. FACADE - Abylaikhan
MainFrame
calls
StudyFacade
+addTask()
+generateSchedule()
ScheduleService
PlanRepository
TaskCreators


7. ADAPTER - Adilzhan
<<interface>>
TaskImporter
+importTasks()
LegacyCsvAdapter
+importTasks()
wraps
OldCsvReader
+readRows()


8. BRIDGE - Adilzhan
StudyReport
+create()
has a
<<interface>>
ReportFormat
+write()
SummaryReport
DetailedReport
PlainTextFormat
HtmlFormat

STUDY SPRINT | UML SEQUENCE DIAGRAM - GENERATE SCHEDULE



Solid arrow: method call | Dashed arrow: return/result